TX – SiMDE : Gestion de l'infrastructure informatique



Réalisé par :

- Yifan ZHANG
- Youssef MOUNHI

Porteur de projet :

- Quentin ANIERE

Remerciements..... 2
Introduction..... 4
Sujet..... 4
Objectifs..... 4
Critères de réussite..... 4
Méthode de travail..... 5
Architecture..... 6
Vue d'ensemble..... 6
Routage et entrée HTTP..... 7
Stockage persistant..... 7
Outils utilisés..... 7
Déploiement des applications..... 8
Objectif et principes généraux..... 8
Étude de cas : Shwet (Laravel) — déploiement Kubernetes (Traefik + Longhorn)..... 8
Vue d'ensemble..... 8
Rôle des manifests (déploiement YAML "direct")..... 9
Procédure de déploiement (ordre recommandé)..... 13
Migration du déploiement Kubernetes (YAML) vers Helm..... 14
Motivation..... 14
Principe avec Helm..... 14
Structure du chart..... 14
Installation / mise à jour..... 15
État actuel et bénéfices..... 16
Généralisation..... 16
Tests de résilience..... 18
Simulation de panne de noeuds..... 18
Scénario 1 : Arrêt d'un noeud worker..... 18
Scénario 2 : Arrêt d'un control-plane..... 20
Scénario 3 : Variation de charge (scaling / réplication)..... 21
Livrables..... 23
Infrastructure as Code..... 23
Scripts et configurations..... 23
Documentation..... 23
Preuves de tests..... 24
Améliorations possibles..... 24
Mise en place d'un outil de GitOps : Argo CD..... 24
Suppression du serveur Nginx comme point d'entrée..... 25
Conclusion..... 25

Remerciements

Nous souhaitons exprimer notre gratitude à **Monsieur Ahmed Lounis** et au **SiMDE** pour avoir proposé cette TX enrichissante et pour leurs précieux conseils tout au long du projet.

Nous remercions également **Quentin Anière**, membre du SiMDE, pour son accompagnement, sa disponibilité et le partage de ses connaissances, qui nous ont permis de mieux comprendre les enjeux liés à la gestion d'un cluster Kubernetes et au déploiement d'applications associatives. Son soutien a été déterminant pour la réalisation et la réussite de ce projet.

Introduction

Sujet

Le SiMDE (*Service informatique de la Maison des Étudiants*) est une commission du BDE chargée de la gestion et du développement des services informatiques destinés aux différentes associations de l'UTC ne disposant pas des ressources nécessaires pour concevoir, administrer et maintenir leurs propres solutions numériques.

Jusqu'à présent, le SiMDE hébergeait ses services sur les serveurs de la DSI de l'UTC. Toutefois, à la suite de l'acquisition de serveurs physiques dédiés, le SiMDE a lancé cette TX dans l'objectif de reprendre la maîtrise de son environnement d'hébergement. Ce projet vise à mettre en place une infrastructure plus adaptée aux enjeux actuels, notamment en matière de sécurité, de scalabilité, de disponibilité et de maintenabilité des applications, en s'appuyant sur la technologie Kubernetes.

Objectifs

Les objectifs opérationnels de cette TX sont les suivants :

- Acquérir les concepts fondamentaux et les principes de fonctionnement de Kubernetes, afin de comprendre son architecture, ses mécanismes internes et ses usages en environnement de production.

- Exploiter un cluster Kubernetes déployé sur l'infrastructure du SiMDE, comprenant des nœuds *control plane* et des nœuds *workers*.

- Déployer des applications associatives au sein du cluster, en mettant en œuvre des stratégies de réplication et, lorsque nécessaire, des mécanismes de stockage persistant.

- Industrialiser les déploiements afin de réduire les risques d'erreurs manuelles et d'améliorer la reproductibilité, notamment par la standardisation des manifestes, le paramétrage des applications et l'utilisation de *templates* (Helm).

- Valider la résilience du cluster en simulant différents scénarios de panne, tels que l'indisponibilité de nœuds, et en observant les mécanismes de récupération mis en œuvre par Kubernetes (replanification des pods, maintien du service, délais de détection et d'éviction).

- Documenter l'ensemble du travail réalisé et produire des guides d'utilisation et de déploiement, centralisés dans un dépôt Git partagé sur l'environnement GitLab de l'UTC.

Critères de réussite

Les principaux critères de réussite retenus pour ce projet sont les suivants :

- Rétablissement du service HTTP après une panne dans un délai compatible avec le contexte associatif, de l'ordre de la minute.
- Redéploiement automatique des pods sur les nœuds encore disponibles du cluster.
- Maintien du fonctionnement du control plane, avec une API Kubernetes restant accessible malgré la perte d'un nœud de type *control plane*.
- Mise en place d'un stockage persistant et résilient, garantissant l'intégrité et la disponibilité des données malgré les défaillances de nœuds.
- Processus de déploiement simple, robuste et reproductible, limitant les erreurs humaines et facilitant la prise en main par de nouveaux contributeurs.
- Documentation claire, structurée et exploitable, permettant la compréhension de l'architecture, des procédures de déploiement et des mécanismes de résilience mis en œuvre.

Méthode de travail

Notre méthode de travail s'est articulée autour de cycles courts et itératifs, nous permettant d'avancer progressivement tout en validant chaque étape du projet.

Dans un premier temps, nous avons procédé à une analyse des besoins du SiMDE, en prenant en compte les contraintes d'hébergement, la criticité des applications, le niveau de disponibilité attendu ainsi que les exigences en matière de stockage.

Nous avons ensuite participé à la validation du cluster Kubernetes, en vérifiant l'état des nœuds, la connectivité entre les composants et le bon fonctionnement des outils d'administration, notamment `kubectl`.

Le déploiement des applications a été réalisé de manière progressive : nous avons débuté par des applications simples, avant d'intégrer des fonctionnalités plus avancées telles que le stockage persistant et des configurations applicatives plus complexes.

Une phase de standardisation a ensuite été menée afin de factoriser les configurations et de rendre les déploiements reproductibles. Cette étape s'est appuyée sur l'utilisation de *templates*, de scripts et de *charts Helm*, en fonction du périmètre et du niveau de généralisation souhaité.

Nous avons également conduit des tests de résilience, en simulant différents scénarios de panne. Ces tests ont donné lieu à une collecte de preuves (sorties `kubectl`, vérification des endpoints, observation du comportement des services HTTP), suivie d'une analyse des résultats obtenus.

Enfin, un travail de documentation a été réalisé tout au long du projet, comprenant la rédaction des procédures, la présentation des résultats, l'identification des limites de la solution et la proposition de pistes d'amélioration.

Tout au long de la TX, nous avons maintenu une communication régulière avec le porteur du projet, Quentin Anière, principalement via la plateforme Discord, ainsi que lors de réunions organisées afin de clarifier les objectifs et de partager l'avancement de nos travaux.

Architecture

Vue d'ensemble

L'installation et la configuration initiale du cluster Kubernetes ont été réalisées par le **SiMDE**. Pour des raisons liées aux politiques d'accès et de gestion des autorisations, nous n'avons pas pris part à cette phase d'installation.
La distribution **K3s** a été retenue pour ce projet, car elle est particulièrement adaptée à notre cas d'usage. Légère et résiliente, elle intègre par défaut plusieurs composants essentiels au bon fonctionnement d'un cluster Kubernetes, tout en limitant la complexité opérationnelle.

Le cluster est composé des éléments suivants :

- **Trois nœuds de type control plane** : `cp1`, `cp2` et `cp3`, intégrant le stockage distribué **etcd**.
  Ces nœuds jouent également le rôle de *worker nodes*, ce qui signifie qu'ils assurent à la fois les fonctions d'administration du cluster et l'hébergement des applications.

- **Deux nœuds workers** : `wk1` et `wk2`.
  Ces nœuds sont exclusivement dédiés à l'exécution et à l'hébergement des applications déployées sur le cluster.

L'architecture réseau repose sur un **point d'entrée Nginx externe**, situé en dehors du cluster Kubernetes. Ce composant agit comme **reverse proxy** et **load balancer**, en redirigeant le trafic entrant vers les nœuds du cluster.
À l'intérieur du cluster, **Traefik** est utilisé en tant qu'**Ingress Controller**, assurant le routage du trafic HTTP vers les *Services* et *Pods* Kubernetes.

Dans les scénarios testés, la chaîne complète de traitement du trafic est donc la suivante :

Client → Nginx (externe) → nœuds du cluster → Traefik (Ingress) → Service → Pods

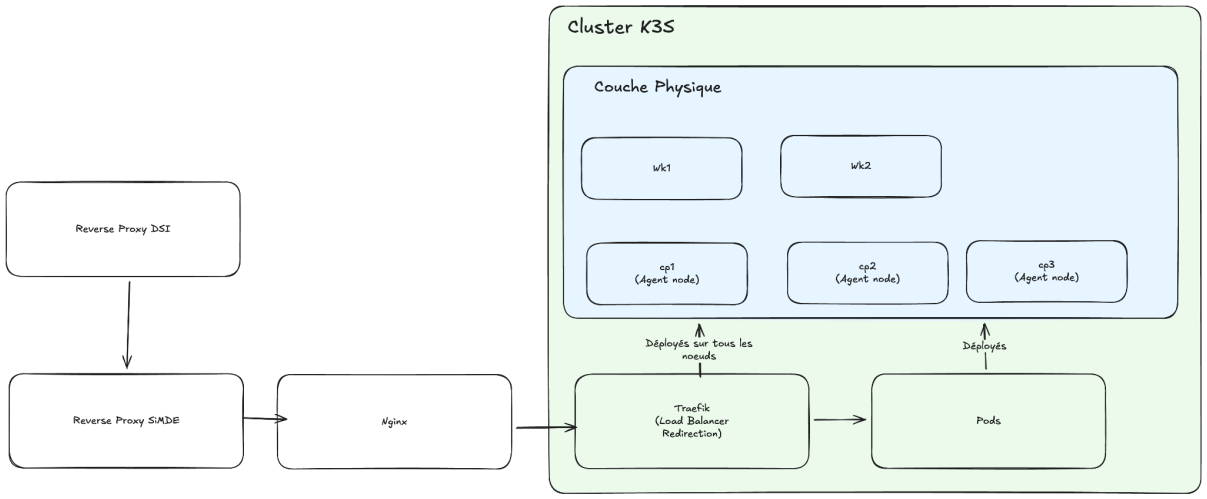


Figure 1: Schéma illustrant l'architecture du cluster K3S

Routage et entrée HTTP

Le trafic HTTP entrant, à l'issue des reverse proxies que nous contrôlons pas, passe d'abord par Nginx, qui redirige ensuite vers les nœuds Kubernetes (extrait de configuration de type `upstream` utilisé dans les tests, montrant plusieurs serveurs backends). Le Load Balancer **Traefik,** installé sur tous les noeuds, gère le trafic au niveau applicatif et le redirige vers le service adapté.

Stockage persistant

Les applications nécessitant de la persistance utilisent un stockage distribué (ex. Longhorn, selon le contexte de tests), afin d'éviter la dépendance à un unique nœud et de permettre le redémarrage/réordonnancement des pods.

Remarque : dans les tests, l'impact du stockage persistant est observé indirectement via la continuité applicative et l'état des pods/volumes.

Outils utilisés

Les principaux outils utilisés tout au long de cette TX sont les suivants :

- **kubectl** : utilisé pour l'administration du cluster Kubernetes, l'observation de son état et le diagnostic des différents composants (nœuds, pods, endpoints, événements).
- **Manifestes Kubernetes et Helm** : employés pour le déploiement des applications, selon les besoins et le niveau de généralisation de chaque application.
- **Nginx** : utilisé comme point d'entrée du cluster dans les scénarios décrits, notamment pour la gestion du trafic entrant.

- **Git** : utilisé pour la gestion de versions et la centralisation de la documentation du projet sur **GitLab**.
- **Discord** : utilisé comme outil de communication et d'échange avec le porteur du projet du SiMD, Quentin Anière.

Déploiement des applications

Objectif et principes généraux

L'un des livrables principaux de cette TX est la **mise en place d'un processus de déploiement reproductible** d'applications associatives sur le cluster Kubernetes du SiMDE. Nous avons suivi les principes suivants :

- **Isolation** par application via un *Namespace* dédié.
- **Exposition HTTP** via un Ingress (Traefik) afin de standardiser l'accès externe.
- **Persistance** des données si nécessaire via PVC (Longhorn).
- **Séparation configuration / secrets** : ConfigMap pour les variables non sensibles, Secret pour les informations sensibles.
- **Opérations applicatives** (ex. tâches planifiées) via CronJob.
- **Reproductibilité** : d'abord en manifests YAML "purs", puis industrialisation avec Helm.

Dans la suite, nous détaillons l'exemple complet de **Shwet (application Laravel)**, car il représente une application réelle avec réseau, stockage persistant et tâches planifiées.

Étude de cas : Shwet (Laravel) — déploiement Kubernetes (Traefik + Longhorn)

Vue d'ensemble

Shwet est déployée dans le namespace `tx-shwet`, exposée via **Traefik Ingress**, avec stockage persistant via **Longhorn**.

- **Namespace** : `tx-shwet`
- **Image** : `registry.gitlab.utc.fr/simde/shwet:latest`
- **Service interne** : `ClusterIP shwet:80` → conteneur `8005`

- **Ingress** : `shwet-tx.assos.utc.fr` (IngressClass `traefik`)
- **Stockage persistant (Longhorn)** :
  - `shwet-documents` (2Gi) → `/var/www/html/storage/app/public/documents`
  - `shwet-logs` (1Gi) → `/var/www/html/storage/logs`
- **Tâches planifiées** :
  - `CronJob shwet-scheduler : */5 * * * * → php artisan schedule:run`

Cette structure est représentative de la plupart des applications web : une partie stateless (pods) + une exposition HTTP + éventuellement une partie stateful (volumes).

Rôle des manifests (déploiement YAML “direct”)

Le déploiement initial a été réalisé via des fichiers YAML, afin de maîtriser précisément le comportement Kubernetes et de produire une base claire avant migration Helm.

00-namespace-config.yaml

Contient **2 ressources** :

- **Namespace `tx-shwet`** : isole toutes les ressources Shwet (Deployment, Service, Ingress, PVC, CronJob…).
- **ConfigMap `shwet-env`** : variables **non sensibles** (exemples) :
  - Laravel : `APP_NAME`, `APP_ENV`, `APP_DEBUG`, `APP_URL`, `LOG_CHANNEL`, `LOG_LEVEL`
  - Base de données (sans mdp) : `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`
  - API : `UTCRAWL_API_URL`
  - Chemins / stockage : `DOCUMENTS_STORAGE_PATH`, `OLD_DATA_STORAGE_PATH`, etc.

    - OAuth : `OAUTH_URL`, `OAUTH_REDIRECT_URI`, `OAUTH_SCOPES`, etc.
    - Drivers : `CACHE_DRIVER`, `QUEUE_CONNECTION`, `SESSION_DRIVER`, …
    - Mail (partie non sensible) : `MAIL_HOST`, `MAIL_PORT`, `MAIL_FROM_ADDRESS`, …
    - Front : `VITE_APP_NAME`, `ASSET_URL`

01-secret.yaml

- **Secret `shwet-secret`** : informations sensibles :
    - `APP_KEY`
    - `DB_PASSWORD`
    - `UTCRAWL_API_KEY`
    - `OAUTH_CLIENT_ID`, `OAUTH_CLIENT_SECRET`
    - `MAIL_USERNAME`, `MAIL_PASSWORD`

Recommandation : en production, éviter de versionner des secrets en clair (ex. SealedSecrets, ExternalSecrets, ou injection au déploiement CI/CD).

05-phpfpm-config.yaml

- **ConfigMap `shwet-phpfpm`** : fournit un `www.conf` pour PHP-FPM (pool `[www]`) monté via `subPath` :
    - Mount : `/etc/php83/php-fpm.d/www.conf`

50-longhorn-pvc.yaml

- **2 PVC** (StorageClass `longhorn`) :
    - `shwet-documents` : `2Gi`, `ReadWriteOnce`

- `shwet-logs` : `1Gi`, `ReadWriteOnce`

Ces PVC sont montés par le Deployment pour rendre persistants **documents** et **logs**.

Point d'attention : avec **ReadWriteOnce (RWO)**, un volume ne peut être monté en écriture que par **un seul nœud à la fois**. Cela convient à un déploiement **mono-replica**, mais devient une contrainte si l'on souhaite **plusieurs replicas** écrivant simultanément sur le même stockage. Dans ce cas, il faut soit passer à une approche **RWX**, soit **externaliser** les fichiers (ex. S3/MinIO), soit **séparer** les volumes par replica selon le besoin.

10-deployment.yaml

- **Deployment** `shwet` (replicas = 1 dans le déploiement initial) :
  - `imagePullPolicy: Always`
  - commande : `sh -c /var/www/html/laravel-setup.sh` (initialisation Laravel)
  - port conteneur : `8005`
  - `envFrom` : `ConfigMap shwet-env` + `Secret shwet-secret`
  - **readinessProbe** : `GET /` sur `8005` (ex. `initialDelaySeconds: 40`)
  - **resources** :
    - requests : `100m / 256Mi`
    - limits : `500m / 512Mi`
  - **volumes / mounts** :
    - config PHP-FPM → `/etc/php83/php-fpm.d/www.conf` (subPath)
    - `shwet-documents` → `/var/www/html/storage/app/public/documents`
    - `shwet-logs` → `/var/www/html/storage/logs`

- `securityContext` : exécution en root (selon contraintes de l'image / permissions)

20-service.yaml

- **Service `shwet`** (ClusterIP) :
  - port `80` (http) → `targetPort: 8005`
  - selector : `app=shwet`

30-ingress.yaml

- **Ingress `shwet`** :
  - `ingressClassName: traefik`
  - host : `shwet-tx.assos.utc.fr`
  - path `/` (Prefix) → `Service shwet:80`

40-cronjob.yaml

- **CronJob `shwet-scheduler`** :
  - schedule : `*/5 * * * * → php artisan schedule:run.`
    Ce pas de 5 minutes a été choisi pour limiter la charge dans le cadre de la TX ; en production, une exécution toutes les minutes est généralement recommandée pour respecter la granularité des tâches Laravel.
  - `concurrencyPolicy: Forbid` (évite chevauchement)
  - historiques : `successfulJobsHistoryLimit: 3`, `failedJobsHistoryLimit: 3`
  - nettoyage : `ttlSecondsAfterFinished: 300`
  - conteneur :

- image : `registry.gitlab.utc.fr/simde/shwet:latest`
- commande : `cd /var/www/html && php artisan schedule:run`
- envFrom : `shwet-env` + `shwet-secret`

Procédure de déploiement (ordre recommandé)

L'ordre d'application est important, notamment pour garantir que les PVC existent avant le démarrage du Deployment.

```
# 1) Namespace + ConfigMap
kubectl apply -f 00-namespace-config.yaml

# 2) Secret
kubectl apply -f 01-secret.yaml

# 3) PVC Longhorn (à créer avant le Deployment)
kubectl apply -f 50-longhorn-pvc.yaml

# 4) Config PHP-FPM
kubectl apply -f 05-phpfpm-config.yaml

# 5) App + réseau
kubectl apply -f 10-deployment.yaml
kubectl apply -f 20-service.yaml
kubectl apply -f 30-ingress.yaml

# 6) Cron
kubectl apply -f 40-cronjob.yaml
```

Vérifications post-déploiement :

```
kubectl -n tx-shwet get pods -o wide
kubectl -n tx-shwet get svc,ingress
kubectl -n tx-shwet describe pod <pod>
kubectl -n tx-shwet get pvc
kubectl -n tx-shwet get cronjob
```

Migration du déploiement Kubernetes (YAML) vers Helm

Motivation

Le déploiement en manifests YAML fonctionne, mais présente des limites en contexte multi-applications / multi-environnements :

- maintenance difficile à long terme,
- duplication de configuration,
- gestion manuelle des secrets et du versioning,
- déploiement moins standardisé.

L'objectif de la migration était de rendre le déploiement :

- **modulaire**, **maintenable**, **réutilisable**,
- **reproductible** et **rollbackable** (historique de releases),
- plus clair en séparant paramètres et secrets.

Principe avec Helm

Le déploiement repose désormais sur un **Helm chart** générant dynamiquement :

- Deployment, Service, Ingress (Traefik)
- ConfigMap (variables non sensibles)
- Secret (variables sensibles)
- PVC Longhorn
- CronJob (scheduler Laravel)
- labels et métadonnées Helm pour suivre les releases

Structure du chart

```
infra/helm/charts/shwet/
├── Chart.yaml
```

```
├── values.yaml
├── values-secret.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── pvc.yaml
│   ├── cronjob.yaml
│   ├── configmap-env.yaml
│   ├── phpfpm-configmap.yaml
│   ├── secret.yaml
│   └── _helpers.tpl
```

- `values.yaml` : configuration applicative & infra (image, ports, replicas, resources, ingress host, paramètres Laravel…)
- `values-secret.yaml` : secrets (exclu du dépôt)
- `templates/` : templates Helm

Installation / mise à jour

```
helm upgrade --install shwet infra/helm/charts/shwet \
  -n tx-shwet \
  -f infra/helm/charts/shwet/values.yaml \
  -f infra/helm/charts/shwet/values-secret.yaml
```

Helm prend en charge :

- installation initiale,
- mises à jour incrémentales,
- historique des releases,
- rollback rapide si nécessaire.

Vérifications Helm utiles :

```
helm -n tx-shwet list
helm -n tx-shwet status shwet
```

```
kubectl -n tx-shwet get all
```

État actuel et bénéfices

- toutes les ressources Kubernetes sont gérées par Helm,
- volumes persistants montés correctement,
- l'application démarre correctement,
- CronJob Laravel fonctionnel,
- accès Traefik opérationnel.

Bénéfices :

- déploiement **standardisé**, **reproductible**,
- séparation claire code / config / secrets,
- mise à jour et maintenance facilitées,
- base réutilisable pour d'autres services.

Généralisation

À l'issue de l'étude de cas menée sur l'application **Shwet**, nous disposions d'une base de configuration Helm standardisée et fonctionnelle. Nous avons donc choisi de nous appuyer sur cette base afin de la **généraliser** et d'**étendre la chart Helm** existante, dans le but d'offrir davantage de paramètres de configuration et de prendre en charge plusieurs types d'applications.

Dans cette optique, nous avons développé une **chart Helm générique**, capable de s'adapter à différents contextes applicatifs. Nous y avons également intégré une **subchart Redis**, proposée comme dépendance optionnelle que le développeur peut activer ou non selon les besoins de son application. Cette instance Redis, déployée directement sur Kubernetes, est principalement destinée à des usages de **test** ou à des besoins simples de **mise en cache**. Néanmoins, pour un usage en production, il est recommandé de recourir à un **service Redis externe**, mieux adapté aux exigences de performance, de disponibilité et de maintenance.

Afin de valider cette phase de généralisation, la nouvelle chart a été testée sur deux applications distinctes :

- **Portail des assos**, une application *Laravel* dont la configuration diffère de celle de Shwet
- **Site vitrine du SiMDE**, une application *Angular*.

Ces tests ont permis de confirmer la flexibilité et la réutilisabilité de la chart générique dans des contextes applicatifs variés.

Dans le cadre de cette étape de généralisation, nous avons produit les livrables suivants :

- une **chart Helm** `generic-app`, intégrant une **subchart Redis** ;
- un **exemple de fichier de valeurs Helm par défaut** (`values.yaml`) correspondant à la nouvelle chart ;
- des **configurations de conteneurs Docker** adaptées aux deux applications ;
- des **scripts de déploiement** ;
- des **fichiers** `README.md` détaillant l'utilisation et la configuration de la chart.

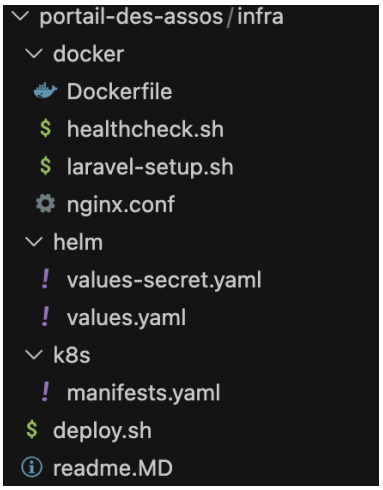


Figure 2 : structure de dossier d'infrastructure de l'application **portail des assos**

```bash
#!/bin/bash
set -e

APP_NAME="portail-des-assos"
NAMESPACE="portail-des-assos"
CHART_PATH="/home/mounhiyo/tx-kubernetes-a25/charts/generic-app"
IMAGE="youssef129/portail-des-assos:latest"

echo "Building Docker image $IMAGE..."
docker build -t $IMAGE -f infra/docker/Dockerfile .

echo "Push the Docker image Docker to the registry..."
docker push $IMAGE

echo "📦 Deploying $APP_NAME..."

# Inject .env to infra/helm/values-secret.yaml
bash $CHART_PATH/scripts/inject-env-in-values.sh

# Deploy
echo "Creating manifests"
helm template $APP_NAME $CHART_PATH \
  -f ./infra/helm/values.yaml -f ./infra/helm/values-secret.yaml \
  -n $NAMESPACE > ./infra/k8s/manifests.yaml
echo "Applying manifests..."
kubectl apply -f infra/k8s/manifests.yaml -n $NAMESPACE

echo "✅ Deployed successfully!"
echo ""
kubectl get pods -n $NAMESPACE
```

Figure 3 : exemple de script de déploiement de l'application **portail des assos**

Tests de résilience

Simulation de panne de noeuds

Pour ce test, nous avons simulé une panne des différents nœuds constituant le cluster kubernetes afin de vérifier son fonctionnement en les arrêtant de manière soudaine.
Avant chaque scénario, nous avons sauvegardé l'état du cluster afin de vérifier les conséquences de chaque test à l'aide des commandes suivantes :

```
$ kubectl get nodes -o wide
$ kubectl get pods -A -o wide
$ kubectl get endpoints -A
```

Scénario 1 : Arrêt d'un noeud worker

Nous allons prendre exemple de l'application *site-vitrine-simde* qui est déployé sur le worker 1 (*wk1*) et worker 2 (*wk2*) :

```
$ kubectl get pods -A -o wide
NAMESPACE                                                         NAME
READY    STATUS       RESTARTS             AGE      IP              NODE
NOMINATED NODE   READINESS GATES
...
site-vitrine-simde        site-vitrine-simde-deployment-6fffc5848d-8nzxx
1/1      Running      0                      15h      10.42.5.160    wk2
<none>           <none>
site-vitrine-simde        site-vitrine-simde-deployment-6fffc5848d-ftmhj
1/1      Running      0                      15h      10.42.4.81     wk1
<none>           <none>
...
```

Remarques :

- Une **fenêtre courte (~1 minute) de requêtes en échec / erreurs intermittentes** a été observée avant un retour à un service nominal.

- > 5 min avant que Kubernetes mette fin aux pods déployés sur wk1 et redéploie un autre pod sur un autre noeud disponible (car le déploiement de l'application possède 2 replicas)

- Après avoir relancer wk1, les pods en statut *Terminating* ont été nettoyés.

Explications :

- **Pourquoi wk1 avait un statut Ready après son arrêt?**
  Kubernetes utilise un mécanisme de heartbeat pour détecter l'état des nœuds. La détection de panne n'est pas instantanée : Kubernetes se base sur des mises à jour périodiques de l'état des nœuds et sur une période de grâce avant de basculer en *NotReady*. Cela explique le décalage entre l'arrêt réel du nœud et son statut visible côté API.

- **Pourquoi les pods restaient sur wk1 pendant plus de 5 minutes ?**
  Kubernetes ne "recrée" pas immédiatement les pods : lorsqu'un nœud devient NotReady/Unreachable, le contrôleur applique des taints (`node.kubernetes.io/not-ready`, `node.kubernetes.io/unreachable`). Les pods disposent généralement de tolerations par défaut (~300s) sur ces taints ; ils ne sont donc évincés puis replanifiés ailleurs qu'après expiration de cette tolérance, afin d'éviter des reschedulings inutiles en cas d'instabilité réseau temporaire.

Explication de l'indisponibilité de l'application :
Le facteur dominant suite à cette indisponibilité est le serveur Nginx qui sert de point d'entrée au cluster Kubernetes.

```
upstream k8s_nodes_backend {
        server 192.168.144.124 max_fails=1 fail_timeout=2s;
        server 192.168.144.125 max_fails=1 fail_timeout=2s;
```

```
        server 192.168.144.126 max_fails=1 fail_timeout=2s;
        server 192.168.144.127 max_fails=1 fail_timeout=2s;
        server 192.168.144.128 max_fails=1 fail_timeout=2s;
        keepalive 16;
    }
```

L'impact observé provient principalement de la **chaîne d'entrée (Nginx → nœuds → Traefik → Service)** : selon les paramètres de **timeouts (connexion/réponse), la persistance de connexions (keepalive)** et la stratégie de retry, une partie des requêtes peut continuer à cibler un backend devenu injoignable, ce qui crée une **fenêtre d'erreurs** côté utilisateur. Kubernetes (rescheduling/éviction) n'est pas le seul facteur : l'**ajustement explicite** des paramètres Nginx (timeouts/retry/keepalive) permet généralement de réduire cette fenêtre.

Conclusion :
Kubernetes fonctionne correctement et nettoie les pods. La fenêtre d'erreurs d'environ 1 minute reste acceptable dans notre contexte. Le principal axe d'amélioration concerne la chaîne d'entrée Nginx (timeouts/retry/keepalive). Comme le SiMDE prévoit d'éliminer cette couche à terme, l'impact côté utilisateur devrait diminuer.

Scénario 2 : Arrêt d'un control-plane

Objectif : vérifier que le cluster reste opérationnel en cas de perte d'un nœud control-plane (API disponible, etcd quorum, scheduling fonctionnel).

Dans ce scénario nous allons nous assurer que le cluster fonctionne toujours après l'arrêt d'un control-plane. Pour cela nous allons vérifier l'état du cluster avec la commande *kubectl get --raw='/readyz?verbose'* qui renvoie l'état des composants du cluster.

```
$ kubectl --server=https://192.168.144.125:6443 get nodes
NAME   STATUS     ROLES                       AGE   VERSION
cp1    NotReady   control-plane,etcd,master   84d   v1.33.5+k3s1
cp2    Ready      control-plane,etcd,master   23d   v1.33.6+k3s1
cp3    Ready      control-plane,etcd,master   84d   v1.33.5+k3s1
wk1    Ready      <none>                      84d   v1.33.5+k3s1
wk2    Ready      <none>                      84d   v1.33.5+k3s1

$        kubectl        --server=https://192.168.144.125:6443        get
--raw='/readyz?verbose'
...
readyz check passed
```

Remarques :

- Le nœud `cp1` est passé en `NotReady`, mais `cp2` et `cp3` restent `Ready`.

- La commande `/readyz?verbose` confirme que l'API et les composants critiques restent disponibles.

- Les applications déjà déployées restent accessibles **tant que leurs pods ne dépendaient pas du nœud arrêté** (si un pod applicatif tournait sur `cp1`, on retombe sur des effets proches du scénario 1).

Explications :

- Avec **3 control-planes**, le cluster peut tolérer la perte d'un nœud control-plane, car les composants de contrôle (API server / etcd) restent disponibles via les autres nœuds tant que le quorum etcd est maintenu.

- Les workloads applicatifs (pods) continuent de tourner sur les workers ; la perte d'un control-plane n'interrompt pas automatiquement les pods déjà démarrés.

- L'impact principal concerne : la capacité à **gérer** le cluster (API), la capacité à **scheduler** de nouveaux pods si trop de composants sont indisponibles, et la disponibilité indirecte si le point d'entrée (Nginx / LB) envoie le trafic vers un nœud devenu injoignable.

Conclusion :
Le cluster Kubernetes continue de fonctionner correctement malgré l'absence d'un control-plane. La redondance des control-planes apporte une tolérance aux pannes conforme aux attentes.

Scénario 3 : Variation de charge (scaling / réplication)

Objectif : vérifier qu'une augmentation de charge peut être absorbée en augmentant le nombre de réplicas, et que Kubernetes répartit correctement les pods.

Procédure :

1. Vérifier l'état initial (pods, replicas, répartition) :

```
kubectl -n <namespace> get deploy -o wide
kubectl -n <namespace> get pods -o wide
```

2. Augmenter le nombre de réplicas (scaling manuel) :

```
kubectl -n <namespace> scale deploy/<deployment-name> --replicas=4
```

3. Observer la création et la répartition :

```
kubectl -n <namespace> get pods -o wide -w
kubectl -n <namespace> describe deploy/<deployment-name>
```

4. Vérifier la continuité du service :

- Test HTTP (curl / navigateur) pendant et après le scaling.
- Vérifier que les endpoints suivent la montée en charge :

```
kubectl -n <namespace> get endpoints
```

Remarques attendues :

- Les nouveaux pods passent `Pending` → `ContainerCreating` → `Running`.
- Le scheduler répartit les pods sur les nœuds disponibles (dans la limite des ressources / contraintes).
- Le service reste disponible ; la montée en charge n'entraîne pas d'interruption notable (sauf lors d'une contrainte ressource ou d'un point d'entrée mal configuré).

Analyse :

- Le scaling horizontal améliore directement la capacité à servir plus de requêtes si :
    - l'application est **stateless** ou a un état externalisé (DB, cache, storage persistant approprié),
    - le service Kubernetes / ingress répartit correctement les requêtes,
    - les ressources cluster (CPU/RAM) sont suffisantes.

Conclusion :
Le cluster supporte une variation de charge via la réplication. Cela valide la scalabilité horizontale comme mécanisme de base. Une étape suivante naturelle serait d'automatiser (HPA) selon des métriques CPU/RAM si la stack de métriques est disponible.

Livrables

Les livrables de la TX sont organisés en plusieurs catégories et sont publiés sur le dépôt Git du **SiMDE**. Ils couvrent l'ensemble des éléments nécessaires au déploiement, à l'exploitation et à la validation de la solution mise en place.

Infrastructure as Code

Cette catégorie regroupe l'ensemble des fichiers liés directement à l'infrastructure Kubernetes et à sa configuration, conformément à une approche *Infrastructure as Code*.

Elle comprend notamment :

- un **déploiement Kubernetes complet en YAML**, incluant les ressources suivantes : *Namespace, Deployment, Service, Ingress Traefik, PersistentVolumeClaim (Longhorn) et CronJob* ;

- un **chart Helm** dédié à l'application *Shwet* ainsi qu'un **chart générique** réutilisable pour d'autres applications, comprenant les *templates* et les fichiers `values.yaml` et `values-secret.yaml`.

Scripts et configurations

Cette partie regroupe les fichiers permettant la construction des conteneurs et l'automatisation des différentes étapes du déploiement en environnement de production.

Elle inclut :

- des **scripts de déploiement**, de **health check** et d'**initialisation applicative** ;

- les **configurations d'entrée** nécessaires au bon fonctionnement des applications (par exemple : *Dockerfile, Nginx, configuration Laravel*), ainsi que les paramètres associés.

Documentation

La documentation vise à assurer la compréhension, la reproductibilité et la maintenabilité du projet.

Elle comprend :

- un **README de déploiement**, détaillant la transition d'un déploiement en YAML vers Helm ;

- les **procédures d'installation, de mise à jour et de vérification** des applications et du cluster.

Preuves de tests

Afin de valider la robustesse et la résilience de l'infrastructure, des preuves de tests ont été produites.

Elles incluent :

- les **résultats** et **captures d'écran** des trois scénarios de résilience étudiés :
    - indisponibilité d'un *worker node* ;
    - indisponibilité du *control plane* ;
    - montée en charge (*scaling*) des applications.

Améliorations possibles

Nous avons identifié deux axes d'amélioration visant, d'une part, à optimiser le processus de développement et de déploiement des applications, et d'autre part, à renforcer la disponibilité et la résilience du cluster Kubernetes.

Mise en place d'un outil de GitOps : Argo CD

Objectif : automatiser les déploiements et garantir la synchronisation entre l'état déclaré dans les dépôts Git et l'état réel du cluster Kubernetes.

L'adoption d'une approche GitOps, à travers l'outil **Argo CD**, permettrait de considérer Git comme la **source unique de vérité** pour l'ensemble des configurations et des déploiements. Tous les changements apportés aux fichiers de configuration (notamment les `values.yaml` des charts Helm) seraient automatiquement détectés et appliqués au cluster.

Argo CD offre également :

- une **interface graphique** permettant de visualiser l'état des applications et des déploiements ;
- un **mécanisme de rollback simplifié**, facilitant le retour à une version stable en cas de problème ;
- un **historique des changements**, assurant une meilleure traçabilité des évolutions ;
- la **détection des modifications appliquées manuellement** directement sur le cluster, en dehors du flux GitOps.

Sur le plan technique, Argo CD est déployé directement dans le cluster Kubernetes sous la forme d'un **controller**. Il peut être combiné avec des outils de gestion des secrets tels que

Sealed Secrets ou **HashiCorp Vault**, afin de sécuriser les données sensibles.
Enfin, cette approche facilite la gestion de **plusieurs environnements** (développement, staging, production) en assurant une cohérence des déploiements entre chacun d'eux.

Suppression du serveur Nginx comme point d'entrée

Objectif : améliorer la répartition de charge et la disponibilité du cluster au niveau réseau.

Une autre amélioration consisterait à éliminer l'utilisation d'un serveur **Nginx** unique comme point d'entrée du cluster. Ce composant constitue un point de défaillance potentiel et limite la capacité de montée en charge.

À la place, l'utilisation d'un **Load Balancer dédié à Kubernetes**, tel que **MetalLB**, permettrait de répartir la charge réseau directement entre les nœuds du cluster. Cette approche améliore la **haute disponibilité**, réduit les risques liés à un point d'entrée unique et s'intègre plus naturellement à l'architecture Kubernetes.

Conclusion

Cette TX a permis de mettre en place une démarche complète et progressive pour déployer et exploiter des applications associatives du SiMDE sur un cluster Kubernetes, avec une attention particulière portée à la maintenabilité et à la disponibilité.

Dans une première étape, nous avons réalisé un déploiement fonctionnel via des manifests Kubernetes écrits en YAML. Cette approche nous a permis de maîtriser précisément les objets essentiels (Namespace, Deployment, Service, Ingress Traefik, ConfigMap/Secret, PVC Longhorn, CronJob) et de valider l'intégration d'une application réelle, Shwet (Laravel), incluant l'exposition HTTP, la persistance (documents/logs) et l'exécution de tâches planifiées.

Ensuite, afin de répondre aux limites naturelles d'un déploiement "YAML manuel" (duplication, maintenance coûteuse, difficultés de paramétrage multi-environnements, gestion moins standardisée des secrets et des mises à jour), nous avons migré ce déploiement vers une approche Helm. Le chart obtenu permet désormais de générer les ressources Kubernetes via des templates, d'externaliser la configuration via values.yaml, de séparer les paramètres sensibles via values-secret.yaml, et de bénéficier d'un workflow plus robuste pour l'installation, la mise à jour et le rollback.

Enfin, nous avons démontré la résilience de l'infrastructure au travers de trois scénarios représentatifs : (1) panne d'un nœud worker, (2) panne d'un nœud control-plane, (3) variation de charge via scaling. Les tests ont confirmé que le cluster maintient le service disponible grâce à la redondance et au rescheduling Kubernetes, tout en mettant en évidence des points d'attention liés à la chaîne d'entrée (load-balancing) et aux contraintes de stockage (RWO/RWX) lors de l'augmentation du nombre de réplicas.

Au final, le projet aboutit à une base d'hébergement opérationnelle et réutilisable pour le SiMDE : un déploiement Kubernetes maîtrisé, une standardisation Helm adaptée à une

gestion collaborative, ainsi qu'un jeu de tests et de preuves permettant de justifier la disponibilité du service. Les prolongements naturels seraient l'ajout d'observabilité (monitoring, alerting, dashboards), et l'automatisation du déploiement via CI/CD afin de fiabiliser encore davantage le cycle de livraison.

Nous avons trouvé cette TX particulièrement intéressante et formatrice. Elle nous a permis de prendre conscience de la complexité du déploiement en environnement de production, ainsi que des différentes problématiques liées à la garantie de la disponibilité et de la résilience des systèmes. Nous y avons appris à utiliser Kubernetes, un outil très recherché sur le marché du travail, et découvert de nouveaux processus et outils associés, tels que Helm, Traefik et les pratiques GitOps.